

Bài 04:

CÁC LƯỒNG NHẬP XUẤT

Nội dung

- ❑ Luồng nhập/xuất là gì?
- ❑ Luồng byte
- ❑ Luồng ký tự
- ❑ Luồng lọc dữ liệu
- ❑ Luồng đệm đệm dữ liệu

Luồng nhập xuất là gì?

- ❑ Khi đọc/ghi, dữ liệu được di chuyển thành dòng gọi là luồng.
- ❑ Luồng chuyển dữ liệu từ nguồn tới đích: Tập tin, thiết bị, socket, và mạng.
- ❑ Dữ liệu trong luồng có 2 dạng: byte và ký tự.
- ❑ Dữ liệu gửi vào luồng có nhiều kiểu khác nhau tương ứng với các kiểu ngôn ngữ Java hỗ trợ.

Ví dụ luồng nhập xuất cơ bản

```
1  import java.io.IOException;
2
3  public class BasicIO {
4
5      public static void main(String[] args) {
6          byte byearr[] = new byte[255];
7          try {
8              System.err.println("Enter the line of text: ");
9              System.in.read(byearr, 0, 255);
10             System.out.println("The line types was:");
11             String str = new String(byearr);
12             System.out.println(str);
13         } catch (IOException e) {
14             System.err.println("Error");
15         }
16     }
17 }
```

Luồng dữ liệu trong java

- ❑ Java cung cấp package java.io với rất nhiều lớp và hỗ trợ nhiều tính năng.
- ❑ Xét theo loại dữ liệu: Java chia các luồng thành hai loại là luồng **byte (byte stream)** và luồng ký tự (**character stream**). Giữa hai loại dữ liệu này có các lớp trung gian để phục vụ cho việc chuyển đổi.
- ❑ Xét theo thao tác: Java chia các luồng thành hai loại là **input (đọc byte)** và **output (ghi byte)** hoặc **reader (đọc character)** và **writer (ghi character)**. Ngoài ra, Java còn hỗ trợ rất nhiều luồng hỗ trợ cho việc đọc ghi khác.

	Byte streams	Character streams
Source streams	InputStream	Reader
Sink streams	OutputStream	Writer

Luồng byte & luồng ký tự

❑ Luồng byte

- Dữ liệu truyền dạng nhị phân.
- Lớp gốc của byte stream là
 - InputStream
 - OutputStream
 - Cả hai là lớp ảo

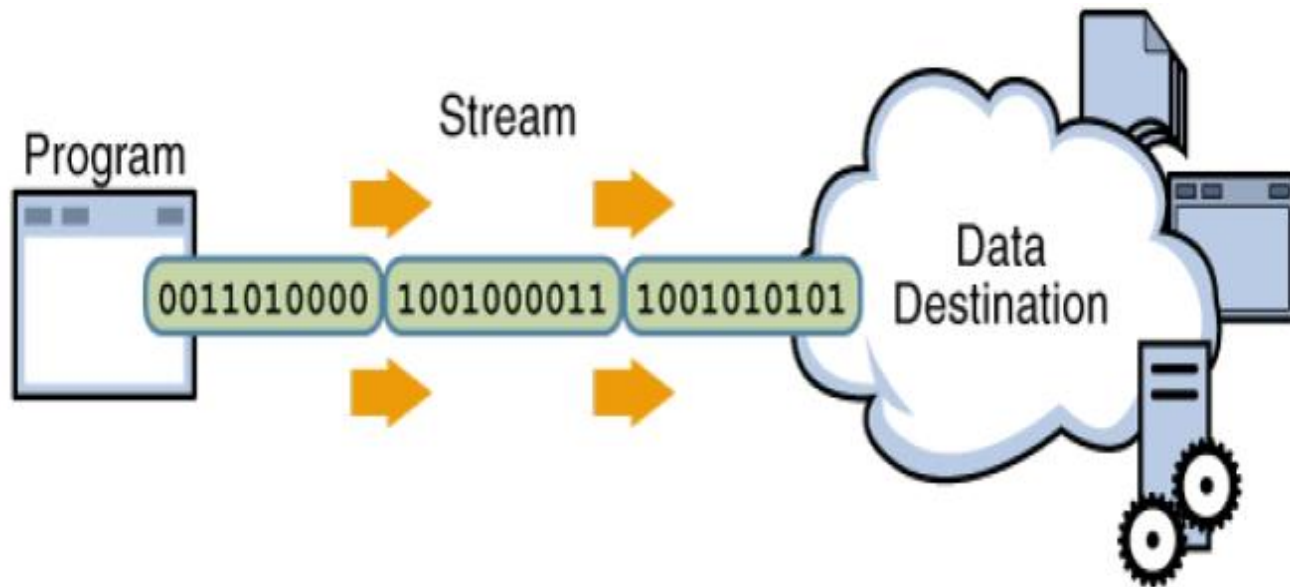
❑ Luồng ký tự

- Dùng với kiểu Unicode
- Lớp gốc của character stream
 - Lớp Reader
 - Lớp Writer
 - Cả hai là lớp ảo

Luồng byte vs luồng ký tự

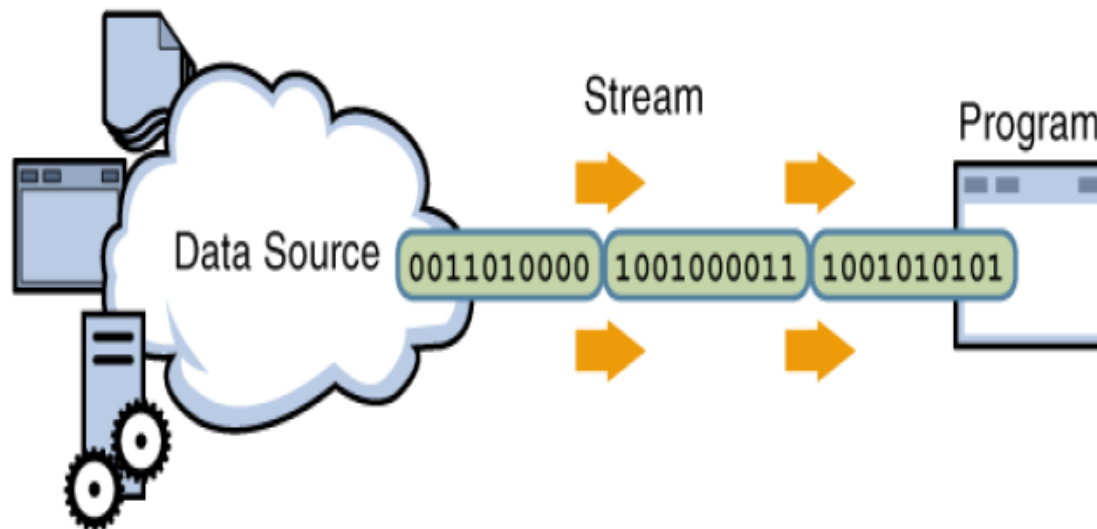
- ❑ Dữ liệu chứa ký tự -> Luồng ký tự
- ❑ Khi nhập, xuất dữ liệu ở mức nhị phân -> Luồng byte
- ❑ Kiểu dữ liệu phức tạp -> Luồng byte

LUỒNG BYTE



Luồng byte đọc

- ❑ Chương trình dùng Input stream để đọc dữ liệu từ nguồn vào chương trình.



Luồng byte đọc

public abstract class **InputStream** implements **Closeable**

InputStream là một lớp nằm trong package java.io, nó là một lớp cơ sở đại diện cho một dòng chảy của các bytes (stream of bytes) có được khi đọc một nguồn dữ liệu nào đó, chẳng hạn file.

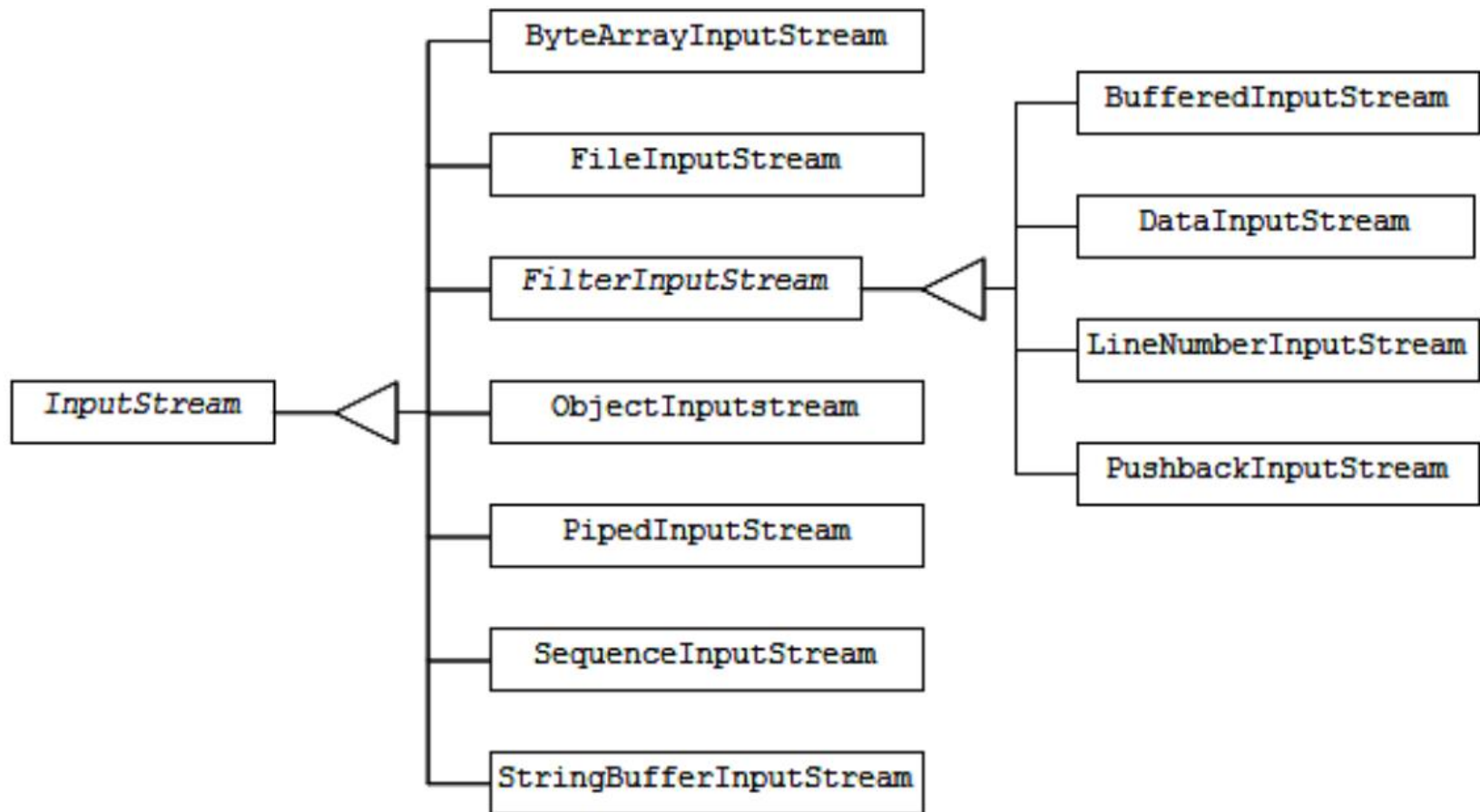
Lớp trừu tượng InputStream

Phương thức	Ý nghĩa
Lớp InputStream	
int available()	Trả về số byte còn lại trong stream.
void close()	Đóng stream.
void mark(int readlimit)	Đánh dấu vị trí hiện tại. Nếu sau khi mark ta đọc quá readlimit byte thì chỗ đánh dấu không còn hiệu lực.
boolean markSupported()	Trả về true nếu stream hỗ trợ mark và reset.
abstract int read()	Đọc byte kế tiếp trong stream. Quy ước: Trả về -1 nếu hết stream.

Lớp trừu tượng InputStream

int read(byte []buffer)	Đọc một dãy byte và lưu kết quả trong mảng buffer. Sau khi đọc xong sẽ trả về số byte đọc được thật sự. Nếu stream đã hết mà vẫn đọc nữa thì kết quả trả về là -1.
int read(byte []buffer, int offset, int len)	Đọc một dãy byte và lưu kết quả trong mảng buffer kể từ byte thứ offset. Số byte đọc được tối đa là len. Trả về số byte đọc được thật sự Nếu stream đã hết thì kết quả trả về là -1.
void reset()	Trở lại chỗ đã đánh dấu bằng phương thức mark().
long skip(long n)	Bỏ qua n byte (để đọc các byte tiếp sau n byte đó).

Các luồng byte đọc hiện thực



Các luồng byte đọc hiện thực

- ❑ `FileInputStream`: cho phép bạn đọc các byte từ tệp tin một cách tuần tự.

Các luồng byte đọc hiện thực

- ❑ `ByteArrayInputStream`: được sử dụng để tạo một luồng đầu vào từ một mảng byte, có nghĩa là điều này cho phép đọc dữ liệu từ mảng byte.

Các luồng byte đọc hiện thực

- ❑ **PipedInputStream**: thường được sử dụng để tạo một kênh truyền thông giữa hai luồng, nơi dữ liệu có thể được gửi từ một luồng (được gọi là "đầu gửi" hoặc "đầu ra") và đọc từ một luồng khác (được gọi là "đầu đọc" hoặc "đầu vào"). Điều này hữu ích trong các tình huống khi muốn truyền dữ liệu trực tiếp từ một luồng đến một luồng khác trong cùng một chương trình.

Luồng byte đọc [BAIO4-VD01]

Chương trình sử dụng `FileInputStream` để đọc dữ liệu từ 1 tập tin trên máy tính sử dụng phương thức `read()`. Với mỗi ký tự đọc được, hiển thị mã ASCII tương ứng.

```
12 import java.io.File;
13 import java.io.FileInputStream;
14 import java.io.IOException;
15 import java.net.MalformedURLException;
16
17 public class FileInputStreamReadEx1 {
18     public static void main(String[] args) throws MalformedURLException, IOException {
19         String path = "C:/test/data.txt";
20         File file = new File(path);
21         FileInputStream fis = new FileInputStream(file);
22         int code;
23         while((code = fis.read()) != -1) {
24             char ch = (char) code;
25             System.out.println(code + " " + ch);
26         }
27         fis.close();
28     }
29 }
```

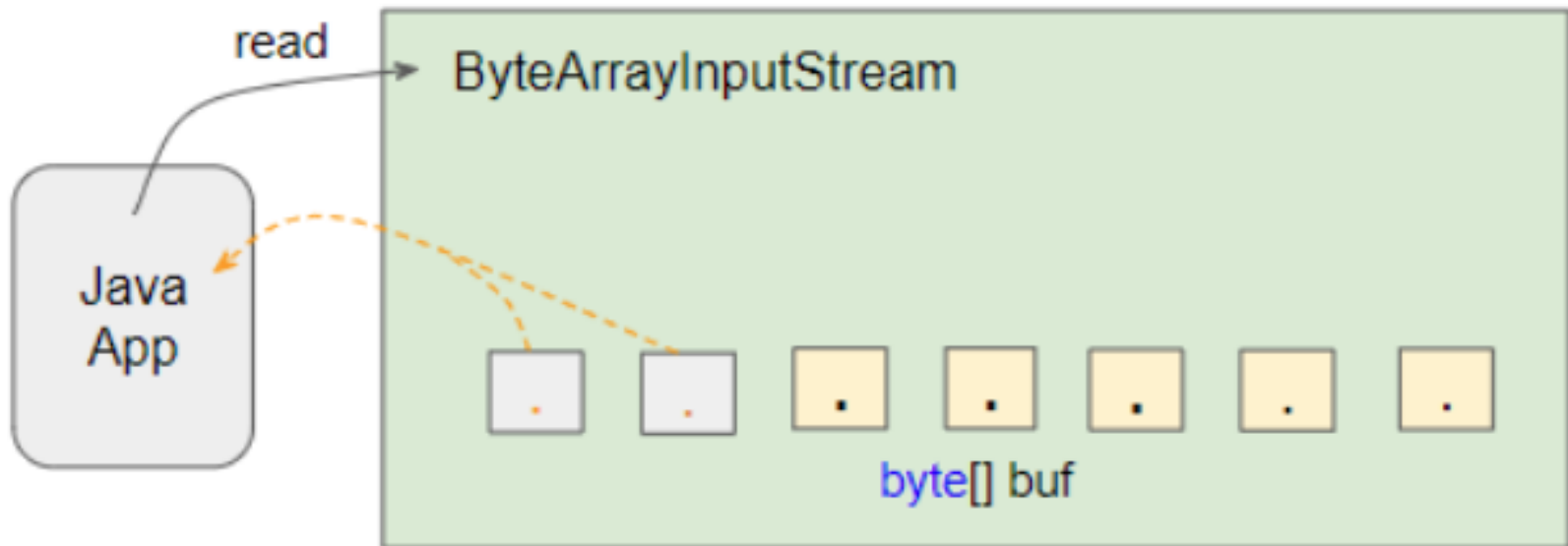
Luồng byte đọc [BAIO4-VD02]

Tương tự như ví dụ 1 nhưng sử dụng phương thức `read(byte[])` để đọc đồng thời nhiều byte từ tập tin.

```
17 public class FileInputStreamReadEx2 {
18     public static void main(String[] args) throws MalformedURLException, IOException {
19         String path = "C:/test/data.txt";
20         File file = new File(path);
21         FileInputStream fis = new FileInputStream(file);
22         // Create a temporary byte array.
23         byte[] tempByteArray = new byte[2];
24         int byteCount = -1;
25
26         int nth = 0;
27         while ((byteCount = fis.read(tempByteArray)) != -1) {
28             nth++;
29             System.out.println("--- Read th: " + nth + " ---");
30             System.out.println(">> Number of bytes read: " + byteCount + "\n");
31
32             for(int i= 0; i < byteCount; i++) {
33                 // bytes are in range [-128,127]
34                 // Convert byte to unsigned byte. [0, 255].
35                 int code = tempByteArray[i] & 0xff;
36
37                 System.out.println(tempByteArray[i] + "    " + code + "    " + (char) code);
38             }
39         }
40         fis.close();
41     }
42 }
```

ByteArrayInputStream

ByteArrayInputStream được sử dụng để đọc một mảng byte theo cách của một InputStream.

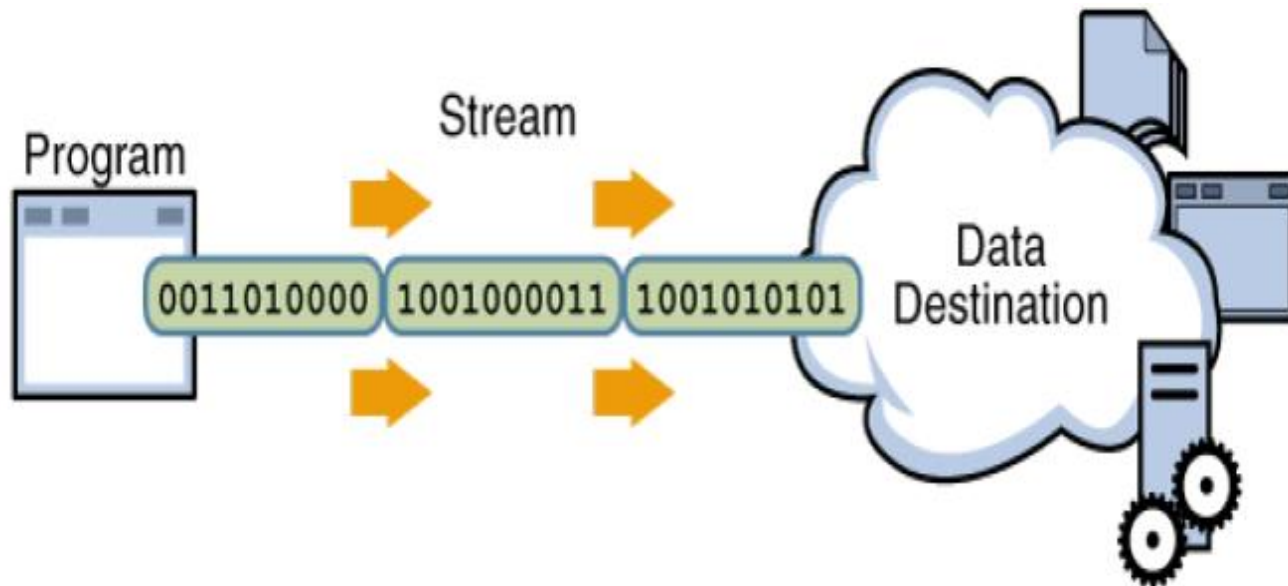


ByteArrayInputStream - ví dụ

```
12 import java.io.ByteArrayInputStream;
13 import java.io.IOException;
14
15 public class ByteArrayInputStreamEx1 {
16
17     public static void main(String[] args) throws IOException {
18
19         byte[] byteArray = new byte[] {84, 104, 105, 115, 32, 105, 115, 32, 116, 101, 120, 116};
20
21         ByteArrayInputStream is = new ByteArrayInputStream(byteArray);
22
23         int b;
24         while((b = is.read()) != -1) {
25             // Convert byte to character.
26             char ch = (char) b;
27             System.out.println(b + " --> " + ch);
28         }
29     }
30 }
```

Luồng byte ghi

- ❑ Chương trình sử dụng output stream để viết dữ liệu từ chương lên đích.



Luồng byte ghi

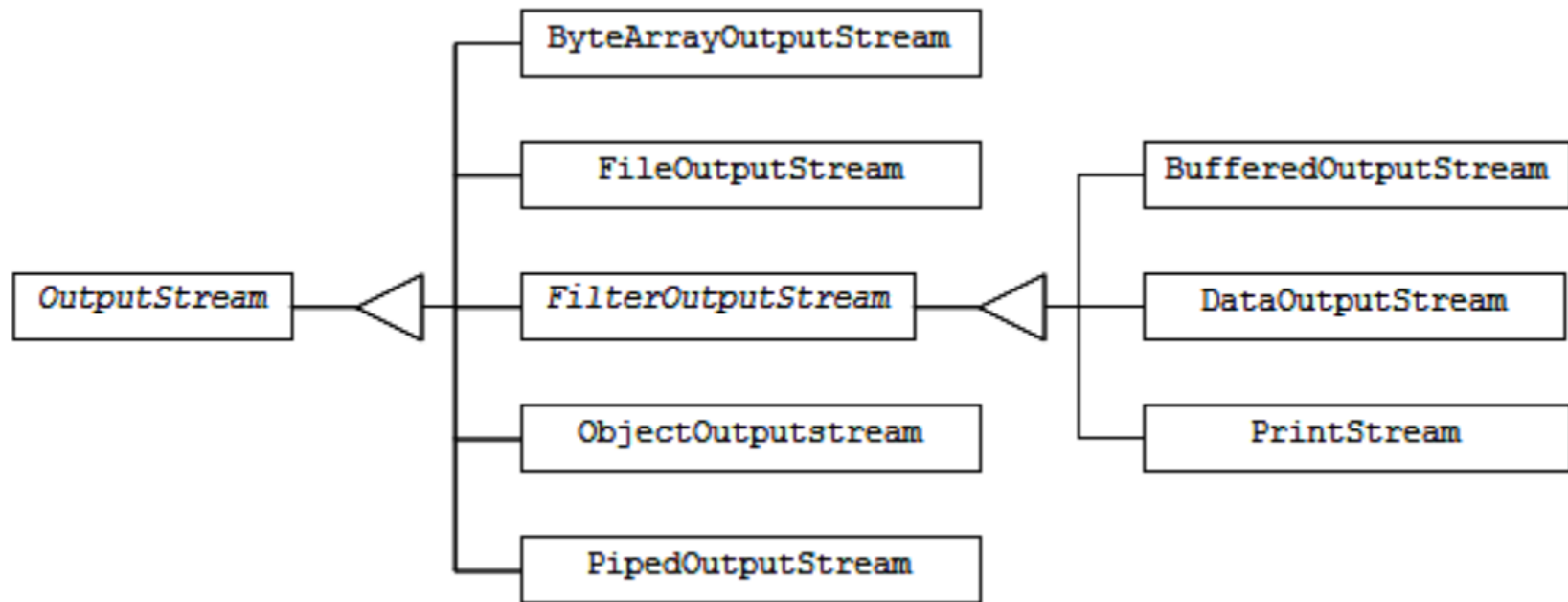
```
public abstract class OutputStream implements Closeable, Flushable
```

OutputStream là một lớp nằm trong package java.io, nó là một lớp cơ sở đại diện cho một dòng chảy của các bytes (stream of bytes) để ghi các bytes vào một mục tiêu nào đó, chẳng hạn file.

Lớp trừu tượng OutputStream

Phương thức	Ý nghĩa
<code>abstract void write(int b)</code>	Ghi một byte vào stream. Đây là phương thức trừu tượng.
<code>void close()</code>	Đóng stream.
<code>void flush()</code>	Buộc stream ghi hết dữ liệu trong vùng đệm ra ngoài.
<code>void write(byte[] b)</code>	Ghi một dãy byte vào stream. Số byte được ghi sẽ là b.length.
<code>void write(byte[] b, int off, int len)</code>	Ghi một dãy byte vào stream. Bắt đầu ghi từ byte thứ off, và ghi tổng cộng len byte

Các luồng byte ghi hiện thực



Các luồng byte ghi hiện thực

- ❑ `FileOutputStream`: ghi (các) byte vào tập tin.
- ❑ `ByteArrayOutputStream`: chứa bộ đệm là mảng byte để ghi dữ liệu.
- ❑ `PipedOutputStream`: ghi (các) byte vào một piped input stream.

Luồng byte ghi [BAIO4-VD03]

Chương trình sử dụng các phương thức của lớp `OutputStream` để ghi chuỗi "ITHUTECH" vào tập tin trên máy tính.

```
17 public class OutputStreamWriteEx1 {
18     // Windows path: C:/Somedirectory/out-file.txt
19     private static final String file_path = "C:/test/data.txt";
20
21     public static void main(String[] args) throws IOException {
22         byte b[]=new byte[20];
23         File file = new File(file_path);
24         // FileOutputStream
25         OutputStream os = new FileOutputStream(file);
26         //Ghi cac byte rieng le
27         os.write(73);
28         os.write((int)'T');
29         //Ghi mang byte
30         String s = "HUTECH";
31         char c[] = s.toCharArray();
32         for(int i = 0; i < c.length; i++){
33             b[i] = (byte)c[i];
34         }
35         os.write(b);
36         // Close
37         os.close();
38     }
39 }
```

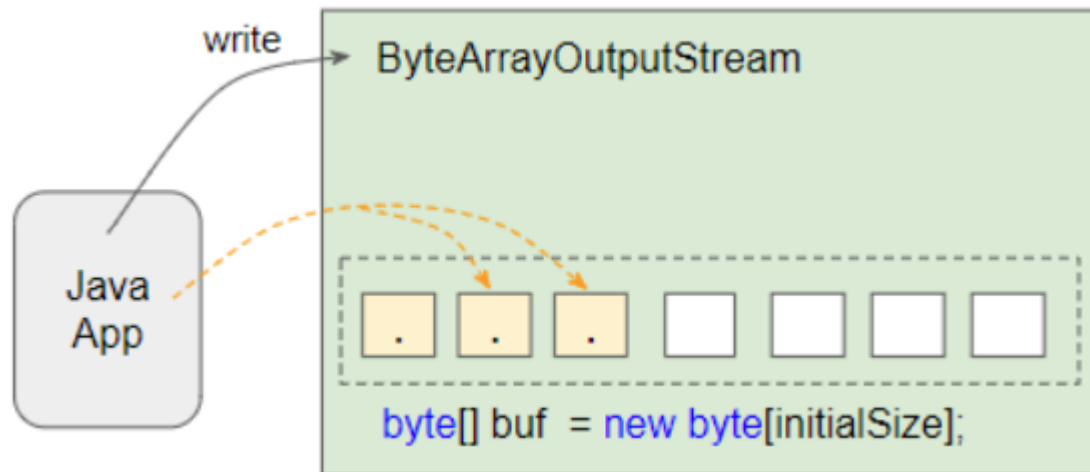
Luồng byte ghi [BAIO4-VD04]

Chương trình sử dụng các phương thức của lớp `OutputStream` để ghi 5 byte từ vị trí 2 vào tập tin.

```
16 public class OutputStreamWriteEx2 {
17
18     public static void main(String[] args) throws IOException {
19
20         // ByteArrayOutputStream is a subclass of OutputStream.
21         OutputStream outputStream = new ByteArrayOutputStream();
22
23         byte[] bytes = new byte[] {'0', '1', '2', '3', '4', '5', '6', '7', '8', '9'};
24
25         outputStream.write(bytes, 2, 5); // '2', ...'6'
26
27         String s = outputStream.toString();
28         System.out.println(s);
29
30         outputStream.close();
31     }
32 }
```

ByteArrayOutputStream

- ❑ `ByteArrayOutputStream` được sử dụng để ghi các bytes vào một mảng byte mà nó quản lý theo cách của một `OutputStream`.
- ❑ Các bytes được ghi vào `ByteArrayOutputStream` sẽ được gán vào các phần tử của mảng byte mà `ByteArrayOutputStream` quản lý.
- ❑ Khi số lượng các bytes ghi vào `ByteArrayOutputStream` lớn hơn độ dài của mảng, `ByteArrayOutputStream` sẽ tạo một mảng mới với độ dài lớn hơn và copy các bytes từ mảng cũ sang.



ByteArrayOutputStream - ví dụ

```
15 public class ByteArrayOutputStreamEx1 {
16
17     public static void main(String[] args) throws IOException {
18         ByteArrayOutputStream baos = new ByteArrayOutputStream(1024);
19
20         byte[] bytes1 = new byte[] {'H', 'e'};
21
22         baos.write(bytes1);
23
24         baos.write(108); // 'l'
25         baos.write('l'); // code: 108
26         baos.write('o'); // code: 111
27
28         byte[] buffer = baos.toByteArray();
29
30         for(byte b: buffer) {
31             System.out.println(b + " --> " + (char)b);
32         }
33     }
34 }
```

Điều khiển xuất nhập sử dụng Stream

Mô hình chung:

- ❑ Tạo ra một đối tượng stream và gắn nó với một nguồn hoặc đích dữ liệu.
- ❑ Cung cấp một vòng lặp để đọc dữ liệu

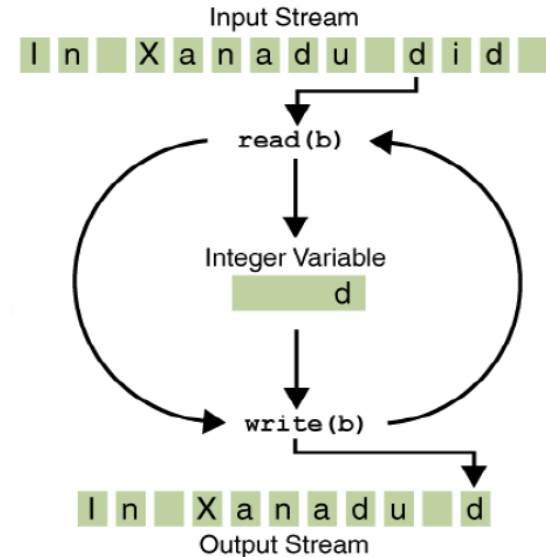
While (còn có thông tin trong luồng)

Đọc (viết) dữ liệu kết tiếp từ (đến) một Stream.

Đóng stream

FileInputStream & FileOutputStream [BAI04-VD05]

```
16 public class FileOutputInputStreamEx1 {
17     public static void main(String [] args) throws IOException{
18         FileInputStream in =null;
19         FileOutputStream out = null;
20         try{
21             in = new FileInputStream("C:/test/inp.txt");
22             out = new FileOutputStream("C:/test/out.txt");
23             int c;
24             while((c=in.read())!=-1)
25                 out.write(c);
26         }finally{
27             if(in != null)
28                 in.close();
29             if((out != null))
30                 out.close();
31         }
32     }
33 }
```



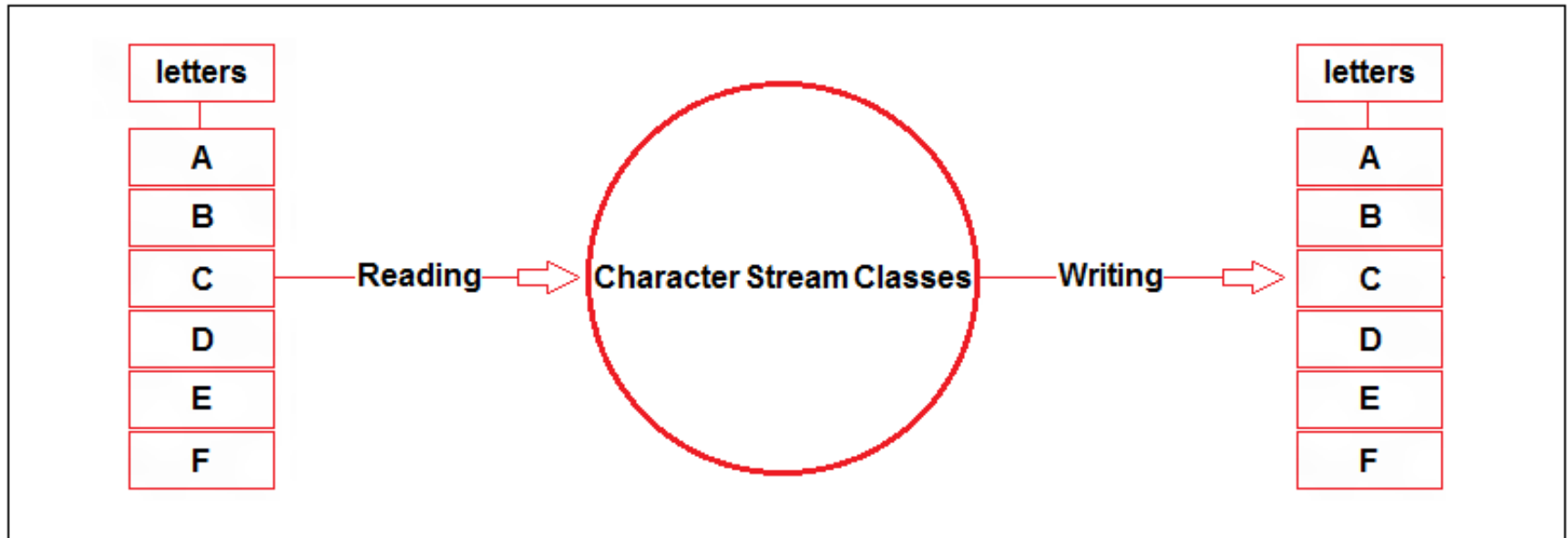
PipedInput Stream và PipedOutputStream

Piped Input Stream	<code>PipedInputStream()</code>	Tạo ra một piped input stream. Stream này chưa được kết nối với piped output stream nào.
	<code>PipedInputStream (PipedOutputStream src)</code>	Tạo ra một piped input stream kết nối với piped output stream src.
	<code>connect (PipedOutputStream src)</code>	Kết nối tới piped output stream src.
Piped Output Stream	<code>PipedOutputStream()</code>	Tạo ra một piped output stream. Stream này chưa được kết nối với piped input stream nào.
	<code>PipedOutputStream (PipedInputStream snk)</code>	Tạo ra một piped output stream kết nối với piped input stream snk.
	<code>connect (PipedInputStream snk)</code>	Kết nối tới piped input stream snk.

Ví dụ PipedInput Stream và PipedOutputStream

```
18 public class PipedOutputInputStreamEx1 {
19     public static void main(String [] args){
20         try{
21             PipedInputStream pi = new PipedInputStream();
22             PipedOutputStream po = new PipedOutputStream(pi);
23             System.out.println("Du lieu ghi:");
24             for(int i=0; i<10; i++){
25                 int b= (int) Math.round(Math.random()*20);
26                 po.write(b);
27                 System.out.println(b+" ");
28             }
29             po.flush();
30             System.out.println();
31             //Doc cac so nguyen tu PipedOutputStream bang PipedInputStream
32             System.out.println("Du lieu doc:");
33             while(pi.available() > 0){
34                 int b=pi.read();
35                 System.out.println(b+" ");
36             }
37             System.out.println();
38             pi.close();
39             po.close();
40         }catch(IOException e){
41             e.printStackTrace();
42         }
43     }
44 }
```

LUỒNG KÝ TỰ



Luồng ký tự

- ❑ Java lưu ký tự sử dụng bảng mã unicode
- ❑ Tất cả các luồng ký tự được kế thừa từ:
 - Reader: Luồng ký tự đọc
 - Writer: Luồng ký tự ghi

Luồng ký tự đọc

`public abstract class Reader implements Readable, Closeable`

Reader là một lớp nằm trong package java.io, nó là một lớp cơ sở đại diện cho một dòng chảy của các ký tự (stream of characters) có được khi đọc một nguồn dữ liệu nào đó, chẳng hạn file văn bản.

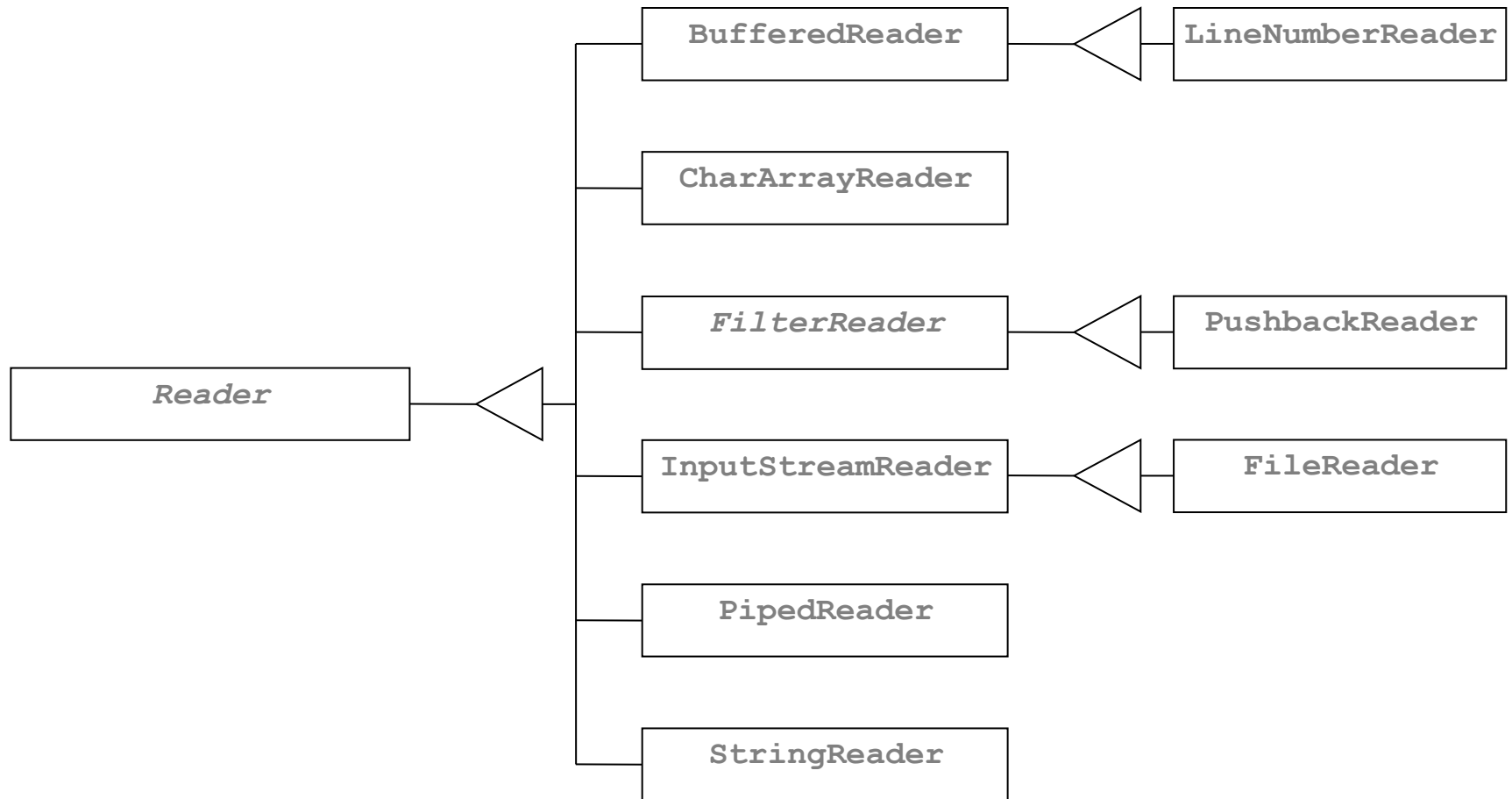
Lớp trừu tượng Reader

Phương thức	Ý nghĩa
Lớp Reader	
void close()	Đóng stream.
void mark(int readlimit)	Đánh dấu vị trí hiện tại. Nếu sau khi đánh dấu ta đọc quá readlimit ký tự thì chỗ đánh dấu không còn hiệu lực.
boolean markSupported()	Trả về true nếu stream hỗ trợ mark và reset.
abstract int read()	Đọc ký tự kế tiếp trong stream. Quy ước: Trả về -1 nếu hết stream. Đây là phương thức trừu tượng.
int read(char [] cbuf)	Đọc một dãy ký tự và lưu kết quả trong mảng cbuf. Sau khi đọc xong sẽ trả về số ký tự đọc được thật sự

Lớp trừu tượng Reader

<code>int read(char[] cbuf, int offset, int len)</code>	Đọc một dãy ký tự và lưu kết quả trong mảng cbuf kể từ ký tự thứ offset. Sau khi đọc xong sẽ trả về số ký tự đọc được thật sự (nếu đang đọc mà hết stream thì số ký tự đọc được thật sự sẽ nhỏ hơn len). Nếu stream đã hết mà vẫn đọc nữa thì kết quả trả về là -1.
<code>void read(CharBuffer target)</code>	Đọc một dãy ký tự và lưu kết quả trong target.
<code>boolean ready()</code>	Trả về true nếu stream sẵn sàng để đọc.
<code>void reset()</code>	Trở lại chỗ đã đánh dấu bằng phương thức mark().
<code>long skip(long n)</code>	Bỏ qua n ký tự (để đọc các ký tự tiếp sau).

Các luồng ký tự đọc hiện thực



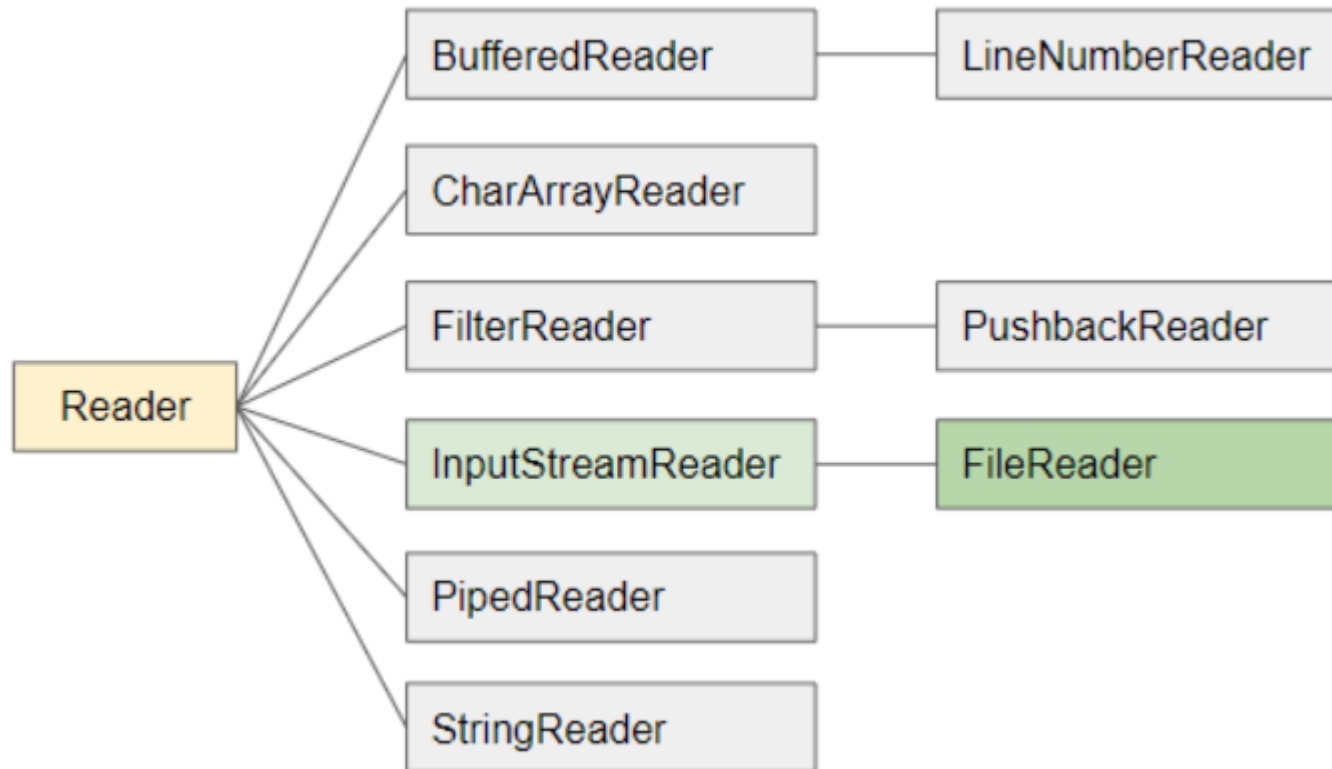
Các luồng ký tự đọc hiện thực

- ❑ `FileReader`: đọc (các) ký tự từ tập tin.
- ❑ `CharArrayReader`: chứa bộ đệm là mảng ký tự để đọc dữ liệu.
- ❑ `PipedReader`: đọc (các) ký tự từ một piped writer.
- ❑ `StringReader`: đọc chuỗi ký tự.

Nhìn chung các lớp này đều có những chức năng chính tương tự nhau.

FileReader

- ❑ FileReader là một lớp con của InputStreamReader, nó được sử dụng để đọc các file văn bản.



FileReader [BAIO4-VD06]

```
17 public class FileReaderEx1 {
18
19     public static void main(String[] args) throws MalformedURLException, IOException {
20         File file = new File("C:/test/data.txt");
21
22         FileReader fis = new FileReader(file);
23
24         int charCode;
25         while((charCode = fis.read()) != -1) {
26             System.out.println((char)charCode + " " + charCode);
27         }
28         fis.close();
29     }
30 }
```

FileReader [BAIO4-VD07]

```
19 public class FileReaderEx2 {
20
21     public static void main(String[] args) throws MalformedURLException, IOException {
22         File file = new File("C:/test/data.txt");
23
24         Reader reader = new FileReader(file);
25          BufferedReader br = new BufferedReader(reader);
26
27         String line;
28         while((line = br.readLine()) != null) {
29             System.out.println(line);
30         }
31         br.close();
32     }
33 }
```

FileReader - ví dụ

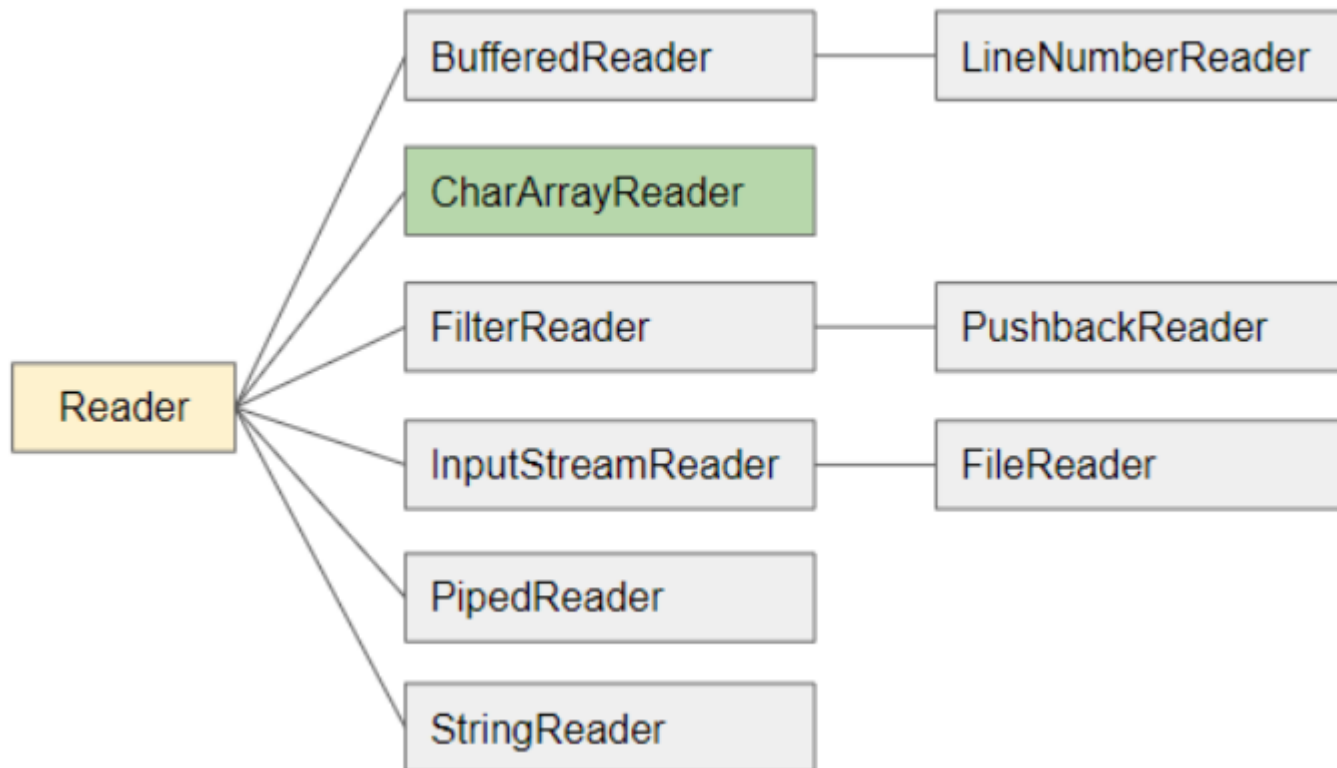
```
17 public class FileReaderEx3 {
18
19     public static void main(String[] args) throws MalformedURLException, IOException {
20         File file = new File("C:/test/students.txt");
21
22         Reader reader = new FileReader(file);
23         BufferedReader br = new BufferedReader(reader);
24
25         br.lines() // java.util.stream.Stream
26             .filter(line -> !line.startsWith("#")) // Not starts with "#".
27             .forEach(System.out::println);
28
29         br.close();
30     }
31 }
```

students.txt - Notepad

File	Edit	Format	View	Help
# Students:				
John P				
Sarah M				
# Sarah B				
Charles B				
Mary T				
Sophia B				

CharArrayReader

- ❑ CharArrayReader là lớp con của Reader. CharArrayReader được sử dụng để đọc một mảng các ký tự theo phong cách của một Reader.



CharArrayReader - ví dụ

```
15 public class CharArrayReaderEx1 {
16
17     public static void main(String[] args) throws IOException {
18
19         char[] charArray = new char[] { 'T', 'h', 'i', 's', ' ', 'i', 's', ' ', 't', 'e', 'x', 't' };
20
21         CharArrayReader reader = new CharArrayReader(charArray);
22
23         int code;
24         while((code = reader.read()) != -1) {
25             char ch = (char) code;
26             System.out.println(ch);
27         }
28     }
29 }
```

CharArrayReader - ví dụ

```
14 public class CharArrayReaderEx2 {  
15  
16     public static void main(String[] args) throws IOException {  
17  
18         String originText = "Log20210217.txt";  
19  
20         char[] charArray = originText.toCharArray();  
21  
22         CharArrayReader reader = new CharArrayReader(charArray);  
23         CharArrayWriter writer = new CharArrayWriter();  
24  
25         int code;  
26         while((code = reader.read()) != -1) {  
27             // '0', '1', ... '9'  
28             if(code >= 48 && code <= 57) {  
29                 writer.write(code);  
30             }  
31         }  
32         char[] newCharArray = writer.toCharArray();  
33         // ['2','0','2','1','0','2','1','7']  
34         System.out.println(newCharArray);  
35  
36         String newText = writer.toString();  
37         System.out.println(newText);  
38     }  
39 }
```

Luồng ký tự ghi

`public abstract class Writer implements Appendable, Closeable, Flushable`

Writer là một lớp nằm trong package java.io, nó là một lớp cơ sở đại diện cho một dòng chảy của các ký tự (stream of characters) để ghi các ký tự vào một mục tiêu nào đó, chẳng hạn file văn bản.

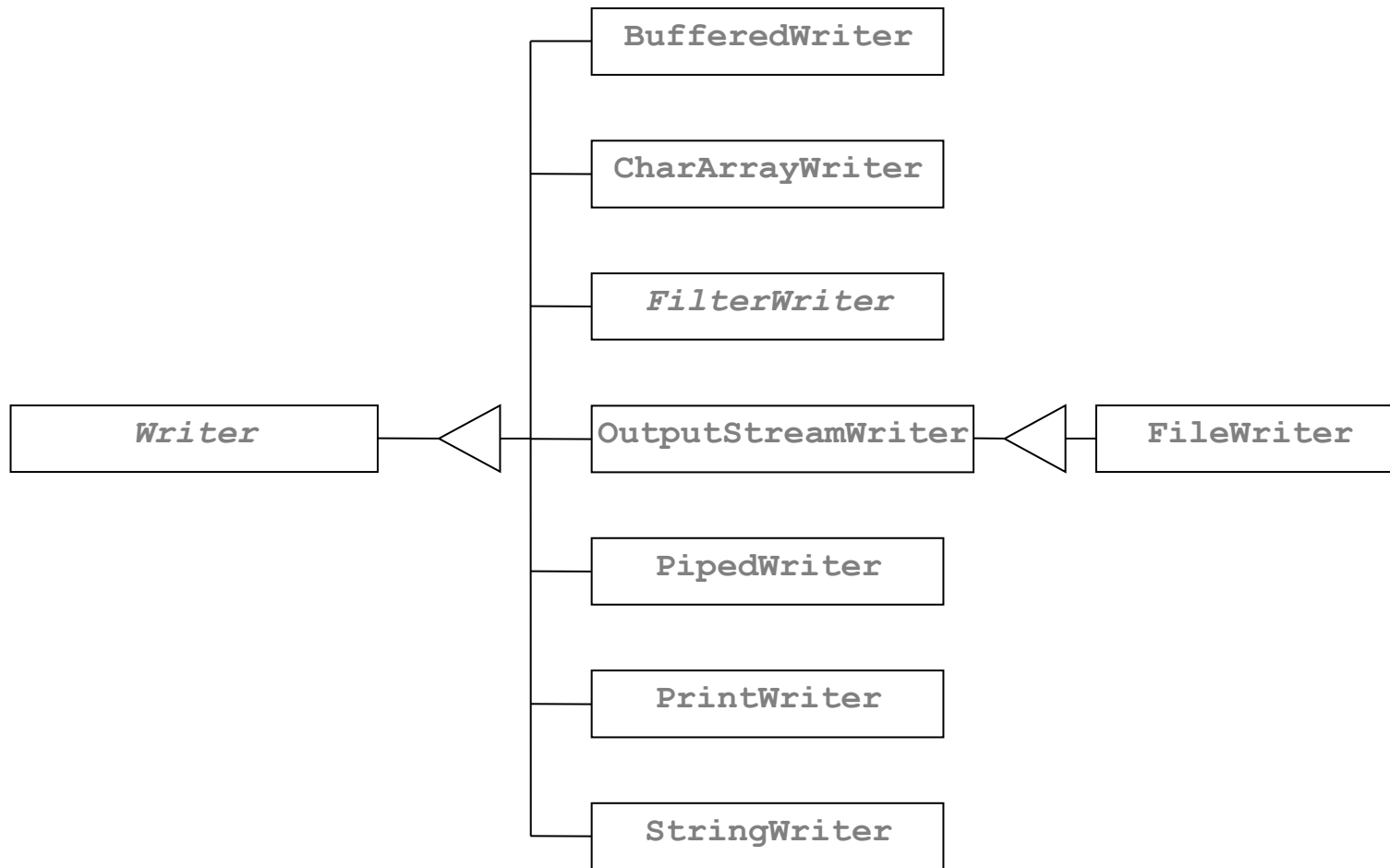
Lớp trừu tượng Writer

Phương thức	Ý nghĩa
<code>Writer append(char c)</code>	Nối đuôi ký tự c vào stream.
<code>Writer append(CharSequence csq)</code>	Nối đuôi dãy ký tự csq vào stream.
<code>Writer append(CharSequence csq, int start, int end)</code>	Nối đuôi một phần của dãy ký tự csq vào stream.
<code>void close()</code>	Đóng stream.
<code>void flush()</code>	Buộc stream ghi hết dữ liệu trong vùng đệm ra ngoài.

Lớp trừu tượng Writer

Phương thức	Ý nghĩa
<code>void write(char[] cbuf)</code>	Ghi một mảng ký tự vào stream. Số ký tự được ghi sẽ là <code>cbuffer.length</code> .
<code>abstract void write(int c)</code>	Ghi một ký tự vào stream. Đây là phương thức trừu tượng.
<code>void write(String c)</code>	Ghi một chuỗi vào stream.
<code>void write(String str, int off, int len)</code>	Ghi một phần của chuỗi vào stream.

Các luồng ký tự ghi hiện thực



Các luồng ký tự đọc hiện thực

- ❑ `FileWriter`: ghi (các) ký tự vào tập tin.
- ❑ `CharArrayWriter`: chứa bộ đệm là mảng ký tự để ghi dữ liệu.
- ❑ `PipedWriter`: ghi (các) ký tự vào một piped reader.
- ❑ `StringWriter`: ghi chuỗi ký tự.

Nhìn chung các lớp này đều có những chức năng chính tương tự như nhau.

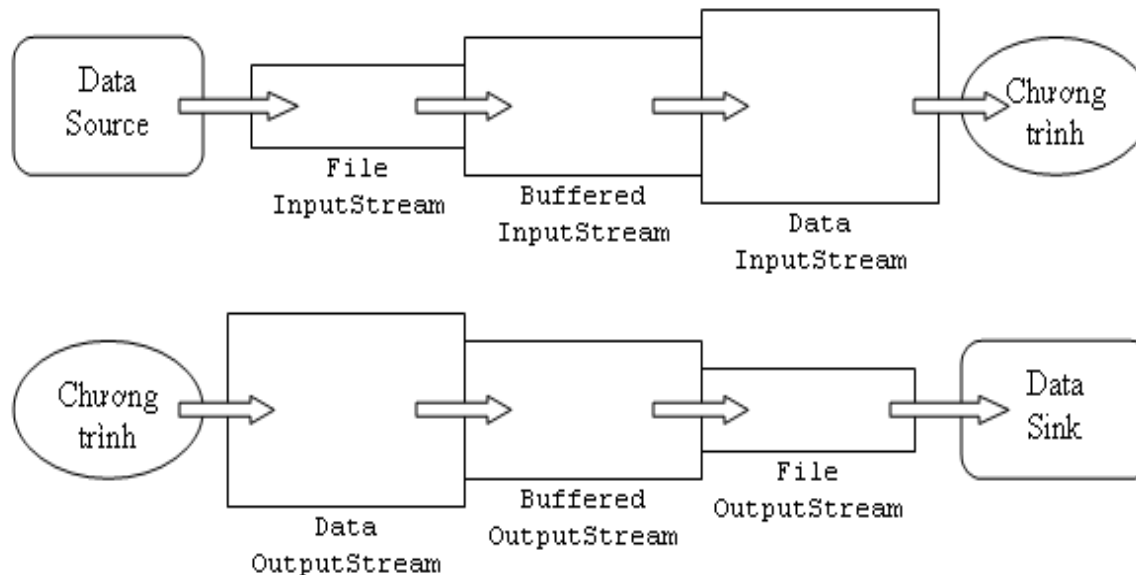
[BAI04-VD08]

```
16 public class ReaderWriterEx1 {
17     public static void main(String[] args) {
18         try{
19             String filename="C:/test/data.txt";
20             String s="Hello file Reader-Writer!";
21             System.out.println("Du lieu ban dau:"+s);
22             //Ghi s vao tap tin
23              FileWriter fw = new FileWriter(filename);
24             fw.write(s);
25             fw.close();
26             //Doc tap tin vao chuoi sb
27             FileReader fr = new FileReader(filename);
28             StringBuffer sb = new StringBuffer();
29             char ca[] = new char[5]; //Doc moi lan 5 ky tu
30             while(fr.ready()){
31                 int len=fr.read(ca); //So ky tu doc duoc that su
32                 sb.append(ca,0,len);
33             }
34             fr.close();
35             //Xuat chuoi sb
36             System.out.println("Du lieu doc duoc:"+sb);
37         }catch(IOException e){
38             System.err.println("Error!");
39         }
40     }
41 }
```

LUỒNG LỌC DỮ LIỆU

Luồng dữ liệu

- ❑ Bởi vì các luồng dữ liệu được chuyển tải dạng byte hoặc ký tự, nên cần có một bộ biến đổi các luồng dữ liệu này sang những giá trị có định kiểu khác nhau.
- ❑ Khi các luồng này được tạo ra, nó phải được gắn với một luồng dữ liệu cụ thể để lọc nó.



Các luồng lọc tổng quát

- ❑ Trong Java, các luồng xử lý lọc tổng quát gồm:
 - `FilterInputStream`: luồng xử lý cho thao tác đọc dữ liệu dạng byte.
 - `FilterOutputStream`: luồng xử lý cho thao tác ghi dữ liệu dạng byte.
 - `FilterReader`: luồng xử lý cho thao tác đọc dữ liệu dạng ký tự.
 - `FilterWriter`: luồng xử lý cho thao tác ghi dữ liệu dạng ký tự.

Các luồng lọc hiện thực

- ❑ Các luồng lọc hiện thực các luồng lọc tổng quát chịu trách nhiệm hiện thực các thao tác biến đổi dữ liệu có định kiểu thành luồng byte/ký tự hoặc ngược lại từ luồng byte/ký tự thành giá trị có định kiểu. Cụ thể, các luồng lọc này là `DataInputStream`, `DataOutputStream`.
- ❑ `DataInputStream`: cho phép đọc các giá trị thuộc kiểu nguyên thủy (**`int`**, **`float`**, **`double`**, **`boolean`**,...).
- ❑ `DataOutputStream`: cho phép ghi các giá trị thuộc kiểu nguyên thủy (**`int`**, **`float`**, **`double`**, **`boolean`**,...).

Các luồng lọc hiện thực

Stream	Phương thức	Ý nghĩa
Data Input Stream	DataStream (InputStream in)	Tạo stream để đọc dữ liệu theo định dạng.
	boolean readBoolean() Các kiểu byte, char, double, float,... cũng có những phương thức tương ứng.	Đọc một giá trị kiểu boolean.
Data Output Stream	DataStream (OutputStream out)	Tạo stream để ghi dữ liệu theo định dạng.
	int size()	Trả về số byte mà stream đã ghi.
	void writeBoolean (boolean v)	Ghi một biến kiểu boolean. Các kiểu byte, char, double, float,... cũng có những phương thức tương ứng.
	void writeBytes(String s)	Ghi chuỗi thành một dãy các byte.
	void writeChars(String s)	Ghi chuỗi thành một dãy các ký tự.

[BAI04-VD09]

```
18 public class DataInputStreamEx1 {
19
20     public static void main(String args[]) throws IOException {
21         //Ghi file có định dạng
22         DataOutputStream dout = new DataOutputStream(new FileOutputStream( name: "file.dat"));
23         dout.writeDouble( v: 1.1);
24         dout.writeInt( v: 55);
25         dout.writeBoolean( v: true);
26         dout.writeChar( v: '4');
27
28         //Đọc file có định dạng
29         DataInputStream din = new DataInputStream(new FileInputStream( name: "file.dat"));
30         double a = din.readDouble();
31         int b = din.readInt();
32         boolean c = din.readBoolean();
33         char d = din.readChar();
34         System.out.println("Values: " + a + " " + b + " " + c + " " + d);
35     }
36 }
37 }
```

BAI04-BT01

Cho bảng dữ liệu sau:

Ngày thực hiện Date	Trạng thái Status	Sản phẩm Item	Số lượng Unit	Đơn giá Price	Tổng tiền Total
2022-09-01	True	Pencil	95	1.99	189.05
2022-09-22		Binder	50	19.99	999.50
2022-10-09	True	Pencil	36	4.99	179.64
2022-11-20		Pen	27	19.99	539.73
2022-12-10	True	Pencil	56	2.99	167.44

Yêu cầu: Thực hiện thao tác ghi các dữ liệu ở bảng trên vào tập tin sau đó đọc dữ liệu từ tập tin và xuất ra màn hình.

LUỒNG ĐỆM DỮ LIỆU

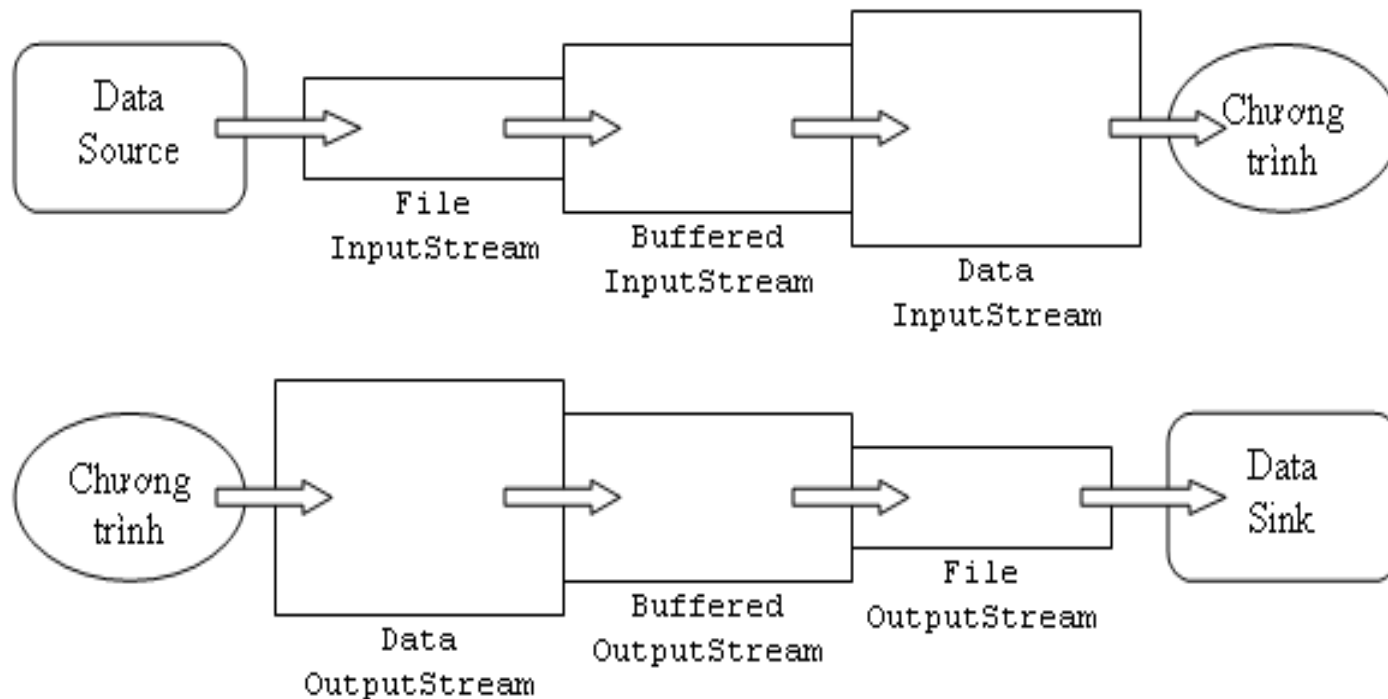
BufferedStream

- ❑ Để tăng tốc độ đọc/ghi → dùng kỹ thuật vùng nhớ đệm.
- ❑ Vùng đệm đọc: dữ liệu sẽ được đọc vào vùng đệm thành từng khối → giảm số thao tác đọc.
- ❑ Vùng đệm ghi: Từng khối dữ liệu được ghi một lần → giảm số thao tác ghi.
- ❑ Như vậy, vùng đệm càng lớn thì càng tăng tốc độ, nhưng sẽ càng dễ xảy ra mất dữ liệu khi chương trình bị chấm dứt đột ngột.

BufferedStream

- ❑ Trong Java, các lớp đọc/ghi theo vùng đệm gồm:
 - `BufferedInputStream`: đọc dữ liệu dạng byte, có dùng vùng đệm
 - `BufferedOutputStream`: ghi dữ liệu dạng byte, có dùng vùng đệm
 - `BufferedReader`: đọc dữ liệu dạng ký tự, có dùng vùng đệm
 - `BufferedWriter`: ghi dữ liệu dạng ký tự, có dùng vùng đệm
- ❑ Tạo stream vùng đệm, ta phải liên kết nó một đối tượng stream khác.

BufferedStream



BufferedStream

Stream	Phương thức	Ý nghĩa
BufferedInputStream	BufferedInputStream (InputStream in, int size)	Tạo một stream với vùng đệm kích thước size để để đọc dữ liệu dạng byte.
BufferedOutputStream	BufferedOutputStream (OutputStream out, int sz)	Tạo một stream với vùng đệm kích thước sz byte để để ghi dữ liệu dạng byte
BufferedReader	BufferedReader (Reader in, int sz)	Tạo một stream với vùng đệm kích thước sz để để đọc dữ liệu dạng ký tự.
BufferedWriter	BufferedWriter (Writer out, int sz)	Tạo một stream với vùng đệm kích thước sz byte để để ghi dữ liệu dạng byte.

Ví dụ về lợi ích việc dùng **BufferedStream [BAI04-VD10]**

```
18 public class BufferedOutputStreamEx1 {
19
20     public static void main(String[] args) throws IOException {
21         String fileName="C:/test/data.txt";
22         long n=50000;
23         try{
24             //Ghi voi stream vung dem
25             long t=System.currentTimeMillis();
26             FileOutputStream fo = new FileOutputStream(fileName);
27             BufferedOutputStream bo = new BufferedOutputStream(fo);
28             for(int i=0; i<n; i++)
29                 bo.write(i);
30             bo.close();
31             t=System.currentTimeMillis()-t;
32             System.out.println("Ghi co vung dem:"+t+" ms");
33             //Khi KHONG co tream vung dem
34             t=System.currentTimeMillis();
35             fo = new FileOutputStream(fileName);
36             for(int i=0; i<n; i++)
37                 fo.write(i);
38             fo.close();
39             t=System.currentTimeMillis()-t;
40             System.out.println("Ghi KHONG co vung dem:"+t+" ms");
41         }catch(IOException e){
42             System.err.println("Error!");
43         }
44     }
45 }
```

Cho biết thời gian thực hiện của 2 luồng trên?

BAI04-BT02

Chương trình mã hóa chuỗi ký tự sử dụng thuật toán mã hóa Caesar.
Trong đó:

- Lớp thứ nhất đọc dữ liệu thô từ tập tin.
- Lớp thứ hai mã hóa dữ liệu và ghi vào file kết quả.

